Tree Traversals

Let's build up from Recursive → Iterative → the O(1) space trick (Morris Traversal).

Each level removes one crutch:
- First you give up manual index tracking (recursion handles it).
- Then you give up the call stack (iterative with your own stack).
- Then you give up the stack entirely (Morris traversal uses the tree's own null pointers as memory).

ROADMAP

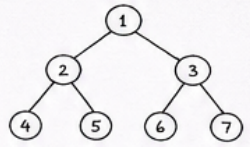
1. Recursive (baseline)
2. Iterative – Preorder
3. Iterative – Inorder
4. Iterative – Postorder
5. One stack – all three
6. Level Order (BFS)
7. Zigzag Level Order
8. Morris Traversal (O(1) space)

1. RECURSIVE (Baseline)

Inorder (L – N – R)

```
def inorderTraversal(root):
    res = []
    def dfs(node):
        if not node: return
        dfs(node.left)
        res.append(node.val)
        dfs(node.right)
    dfs(root)
    return res
```

Inorder Flow



Inorder : [4, 2, 5, 1, 6, 3, 7]

Preorder (N – L – R)

```
def preorderTraversal(root):
    res = []
    def dfs(node):
        if not node: return
        res.append(node.val)
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return res
```

Preorder : [1, 2, 4, 5, 3, 6, 7]

Postorder (L – R – N)

```
def postorderTraversal(root):
    res = []
    def dfs(node):
        if not node: return
        dfs(node.left)
        dfs(node.right)
        res.append(node.val)
    dfs(root)
    return res
```

Postorder : [4, 5, 2, 6, 7, 3, 1]

Time : O(n) Space : O(h) (h = height of tree)
Worst case (skewed tree) ⇒ h = n Balanced tree ⇒ h = log n

2. ITERATIVE – PREORDER (Easiest)

- Preorder visits node first.
- So visit immediately after pop.
- Push Right first, then Left so that Left is popped first (stack is LIFO).

Push order: Right ↓ Left → Left popped first.

```
def preorderTraversal(root):
    if not root: return []
    stack, res = [root], []
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.right: stack.append(node.right)
        if node.left : stack.append(node.left)
    return res
```

Example

Output: [1, 2, 4, 5, 3]

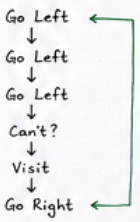
Time : O(n) Space : O(h)

3. ITERATIVE – INORDER (The standard pattern)

- Go as far left as possible, pushing all nodes.
- Pop, visit, then go right.

```
def inorderTraversal(root):
    stack, res = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        res.append(curr.val)
        curr = curr.right
    return res
```

Dry Run Idea



Time : O(n) Space : O(h)

4. ITERATIVE – POSTORDER (The annoying one!)

- Postorder = visit node last.
- Trick: Do (Node → Right → Left) and reverse.

Modified Preorder: Node ↓ Right ↓ Left → Reverse → Postorder: Left ↓ Right ↓ Node

```
def postorderTraversal(root):
    if not root: return []
    stack, res = [root], []
    while stack:
        node = stack.pop()
        res.append(node.val)
        if node.left : stack.append(node.left)
        if node.right: stack.append(node.right)
    return res[::-1]    # reverse
```

Example

Output: [4, 5, 2, 3, 1]

5. ONE STACK – ALL THREE AT ONCE

- Store (node, state).
- Each node goes through 3 states.

```
def preorder_inorder_postorder(root):
    pre, ino, post = [], [], []
    stack = [(root, 0)]
    while stack:
        node, state = stack.pop()
        if not node: continue
        if state == 0:
            pre.append(node.val)
            stack.append((node, 1))
            stack.append((node.left, 0))
        elif state == 1:
            ino.append(node.val)
            stack.append((node, 2))
            stack.append((node.right, 0))
        else:
            post.append(node.val)
    return pre, ino, post
```

State	Meaning
0	Preorder (N)
1	Inorder (N)
2	Postorder (N)

Flow for one node: state 0 ↕ (Pre) → state 1 ↓ (In) → state 2 ↓ (Post)

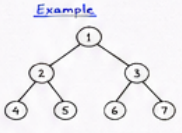
Time : O(n)
Space : O(h)

6. LEVEL ORDER (BFS)

- Uses a Queue (FIFO).
- Process level by level.

```
from collections import deque
def levelOrder(root):
    if not root: return []
    res, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left : queue.append(node.left)
            if node.right: queue.append(node.right)
        res.append(level)
    return res
```

Example



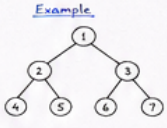
Output: [[1], [2, 3], [4, 5, 6, 7]]

7. ZIGZAG LEVEL ORDER

- Same BFS, but reverse every alternate level.
- Toggle direction after each level.

```
def zigzagLevelOrder(root):
    if not root: return []
    from collections import deque
    res, queue, left_to_right = [], deque([root]), True
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left : queue.append(node.left)
            if node.right: queue.append(node.right)
        res.append(level if left_to_right else level[::-1])
        left_to_right = not left_to_right
    return res
```

Example



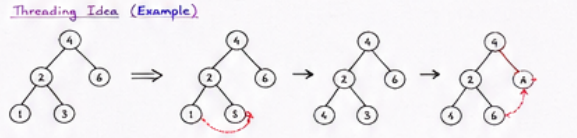
Output: [[1], [3, 2], [4, 5, 6, 7]]

8. MORRIS TRAVERSAL (O(1) SPACE) – INORDER

Big Idea

- Don't use stack or recursion.
- For a node with left child, find its inorder predecessor (rightmost node in left subtree).
- Make predecessor.right point back to current node (thread).
- When thread is seen again, remove it and visit node.

Threading Idea (Example)



At node 4 (has left child) ⇒ Make thread: 3.right → 4 → Go left, visit 1, then 2, then 3 → Thread exists again. Remove it and visit 4.

Algorithm

1. curr = root
2. while curr:
   a) if curr.left is None:
      visit curr
      curr = curr.right
   b) else:
      prev = rightmost of curr.left
      if prev.right is None:
         prev.right = curr (make thread)
         curr = curr.left
      else:
         prev.right = None (remove thread)
         visit curr
         curr = curr.right

Time : O(n) Space : O(1)

Tree structure is restored after traversal.

Why O(1)?

- No stack
- No recursion
- No queue
- We only reuse NULL right pointers temporarily.
- Restored before moving on.

CHEAT SHEET

Preorder	N – L – R
Inorder	L – N – R
Postorder	L – R – N
Level Order	Level by Level (BFS)
Zigzag	Alternate Levels
Morris Inorder	L – N – R (O(1) space)

Remember:
Understand the idea, not just the code.
Once the idea clicks, the code becomes easy! ☺